

Report on ASR model training

Serkov Aleksandr

Faculty Of Informatics, Physics and Technologies, HSE, SPb
amserkov@hse.ru

Last Edit Date: 17 октября 2025 г.

Аннотация—Данная работа посвящена реализации и обучению двух архитектур для задачи ASR: LAS (Listen, Attend and Spell) и Conformer (основанный на CTC-декодировании)

How to reproduce your model?

Достаточно запустить обучение на конфиге `stable` для обучения `conformer` модели или `stable_las` для обучения `las` модели.

Чтобы запустить эксперименты с `bpe`, нужно заменить `CharTokenizer` на `BPETokenizer` в изначальных файлах конфигурации.

How did you train your final model?

Качестве финальной модели была выбрана архитектура `conformer_small`:

```
1 n_feats: 80
2 enc_dim: 144
3 enc_layers: 16
4 enc_heads: 4
5 enc_kernel_size: 31
6 dec_dim: 320
7 dec_layers: 1
```

Обучение было произведено на 80 эпохах по 800 шагов. В ходе него были отключены все аугментации, кроме `gain` для ускорения сходимости. `lr` (10^{-3}), а также другие параметры для оптимизатора были взяты из оригинальной статьи про конформеры. Размер батча был снижен до 16 для ускорения сходимости и баланса по качеству.

Тренировка была проведена на `nvidia RTX 3070 mobile` (почему не `коллаб` или `кагл` - потому что у меня там обучаются модельки для других проектов, да и скорость там не сильно выше) и заняла порядка 8 часов реального времени.

What have you tried?

В ходе экспериментов были попробованы модели `Conformer`, `Conformer-LSTM` (декодер из оригинальной статьи) и `LAS`.

Из аугментаций были попробованы `Pitch shift`, `Bit regressor`, `Gain`, `Time stretch`, `Background noise` и `Spec aug`

What worked and what did not work?

Интересным наблюдением стала неправильная работа `pitch shifting-a` (изначально она была реализована через две интерполяции): результат был очень похож на снижение битности из-за чего возник реализован метод аугментации - `bit regressor`, который снижает битность входного потока (в ходе обучения с ним модель обучалась сильно менее стабильно, но чуть быстрее).

В ходе работы токенизация была вынесена в отдельные модели, для упрощения работы с `bpe`. По наблюдением `bpe` (а именно урезанная до 2000 токенов версия `gpt2 tokenizer`, дабы не жертвовать производительностью на вычисление `ctc-loss`) работает сильно хуже для моделей `ctc_small` и `ctc_medium` несмотря на примечание в задании (дело в том, что мы сильно разряжаем частотность каждого токена из-за чего обучать приходится дольше, а сама модель может быть сильно маленькой для поиска более сложных закономерностей на уровне определённого окна спектрограммы: нам приходится наращивать это окно при росте величины токенов).

Также были внедрены несколько методов улучшения качества инфренса: `beam_search`, `beam_lm` (расчет графа с добавлением вероятностей `kenlm` и штрафа за окончание слова: модель имеет склонность бить слова на куски) и `beam_lm_rescoring` (стандартный `beam_search`, но к итоговым ветвям добавляется вероятность `kenlm`, а затем выбирается лучший луч). По результату промежуточных экспериментов, второй подход показал наилучшие результаты, поэтому именно он использовался для выбора лучшей модели. Также для `las` была создана отдельная версия для генерации с помощью `beam_search`, однако она так и не была использована в силу плохих результатов самой модели.

What were the major challenges?

Основная трудность данной задачи состояла в обучении моделей: каждая из архитектур, каждый из подходов к аугментации, равно как и каждая конфигурация запуска занимала несколько часов (а порой и десятки часов), дабы получить результаты, на основе которых можно было бы внести какие-либо изменения. Поэтому первоочередной задачей стал поиск таких параметров, на которых достигались бы разумные результаты за

сжатые сроки. Таким образом я пришел к архитектуре с конфигурациями `cts_small` и `las`.

Также под нож пошли все аугментации, кроме `gain`, т.к. они сильно замедляли сходимость (по наблюдениям, в 3 – 4 раза). Возможно, они были слишком агрессивные, хотя я отдельно проверил их работу (`testing_augs.ipynb`).

Кроме этого, батчи величиной более 10 не влезали в память из-за чего пришлось реализовывать аккумуляцию градиентов (при батчах с 10-тью элементами и менее сходимости практически не наблюдалось).

Дополнительной трудностью стала отладка архитектур: они содержат большое число тонкостей, которые не упоминаются в статье, либо же описаны лишь в общих чертах. Всё осложнялось тем, что статья не предлагает авторского кода, а те реализации, что есть в интернете либо сильно отходят от оригинала, либо содержат большое число ошибок (в работе не были использованы чужие реализации, однако были произведены сверки с общедоступными реализациями, дабы снизить возможность ошибки).

Так, например, очень долгое время `conformer` никак не мог дойти до хоть сколько-то разумных значений. Дело оказалось в отсутствии позиционного кодирования во внимании.

Analysis

Анализ результатов обучения продемонстрировал заметную разницу в качестве и стабильности между тремя архитектурами: `Conformer-CTS`, `Conformer-CTS (BPE)` и `LAS`.

1. `Conformer-CTS`. На графиках (рис. 1) отчетливо видно устойчивое снижение функции потерь как на обучающем, так и на валидационном и тестовом наборах. Уже к 20k шагу наблюдается стабилизация обучения, а к 60k шагу значения `loss_val` и `loss_test` составляют 0.24 и 0.54 соответственно. При этом метрики `CER` и `WER` демонстрируют устойчивое снижение: `CER (val)` достигает 6.88, а `WER (val)` - 22.32. Использование `CTS`-декодирования обеспечивает хорошее качество при относительно малом количестве параметров и сравнительно коротком времени обучения. Также стоит отметить, что есть также потенциал для дальнейшего обучения (метрики не вышли на плато).

Дополнительные эксперименты с методами `beam search` и `beam search + LM` показали, что использование языковой модели не всегда приводит к улучшению - в данном случае наблюдается даже ухудшение метрик (см. Табл. I), Возможно, имеет место более точная калибровка весов.

2. `Conformer-CTS (BPE)`. Попытка перейти к `BPE`-токенизации (рис. 2) привела к деградации качества: `CER/WER` достигли практически 100%. Анализ кривых показывает отсутствие сходимости - функция потерь остаётся высокой, а значения ошибок не уменьшаются. Это подтверждает предположение о чрезмер-

ной разреженности словаря при ограниченном размере модели: частота появления токенов оказалась недостаточной для эффективного обучения `CTS`-механизма. Таким образом, для успешного применения `BPE` требуется либо более крупная модель, либо увеличенный объём данных и шагов обучения.

3. `LAS`. Архитектура `Listen, Attend and Spell` (рис. 3) показала частичную сходимость, но итоговые метрики (`CER` $\approx$ 57, `WER` $\approx$ 110) указывают на недостаточную обобщающую способность. Несмотря на плавное снижение функции потерь, наблюдается заметный разрыв между `train` и `val` кривыми, что говорит о переобучении. Относительно высокая чувствительность `LAS` к выбору длины последовательности и параметров внимания также могла повлиять на результат. Тем не менее, тенденция к снижению ошибок говорит о потенциальной возможности улучшения за счёт более длительного обучения или регуляризации.

Приложение

A. Итоговые значения моделей

На всех вычислениях был поставлен `early-stopping` на 4 шага. Ширина графа во всех случаях - 10. Вес `lm` - 0.5, бонус за слово - 1.1

B. Процесс обучения итоговой модели

Конфигурация полностью соответствует описанию второй секции

C. Процесс обучения итоговой модел

Конфигурация полностью соответствует финальной модели, за тем лишь исключением, что используется `gpt-2 tokenizer` с урезанным словарём (до 2000 элементов).

D. Процесс обучения LAS модели

Для `LAS` был использован следующий конфиг:

```
1 _target_: src.model.las_model.LasModel
2 n_feats: 80
3 enc_layers: 3
4 enc_hidden: 256
5 dec_layers: 1
6 dec_hidden: 512
7 enc_dropout: 0.1
```

Рис. 1. Процесс обучения итоговой модели.

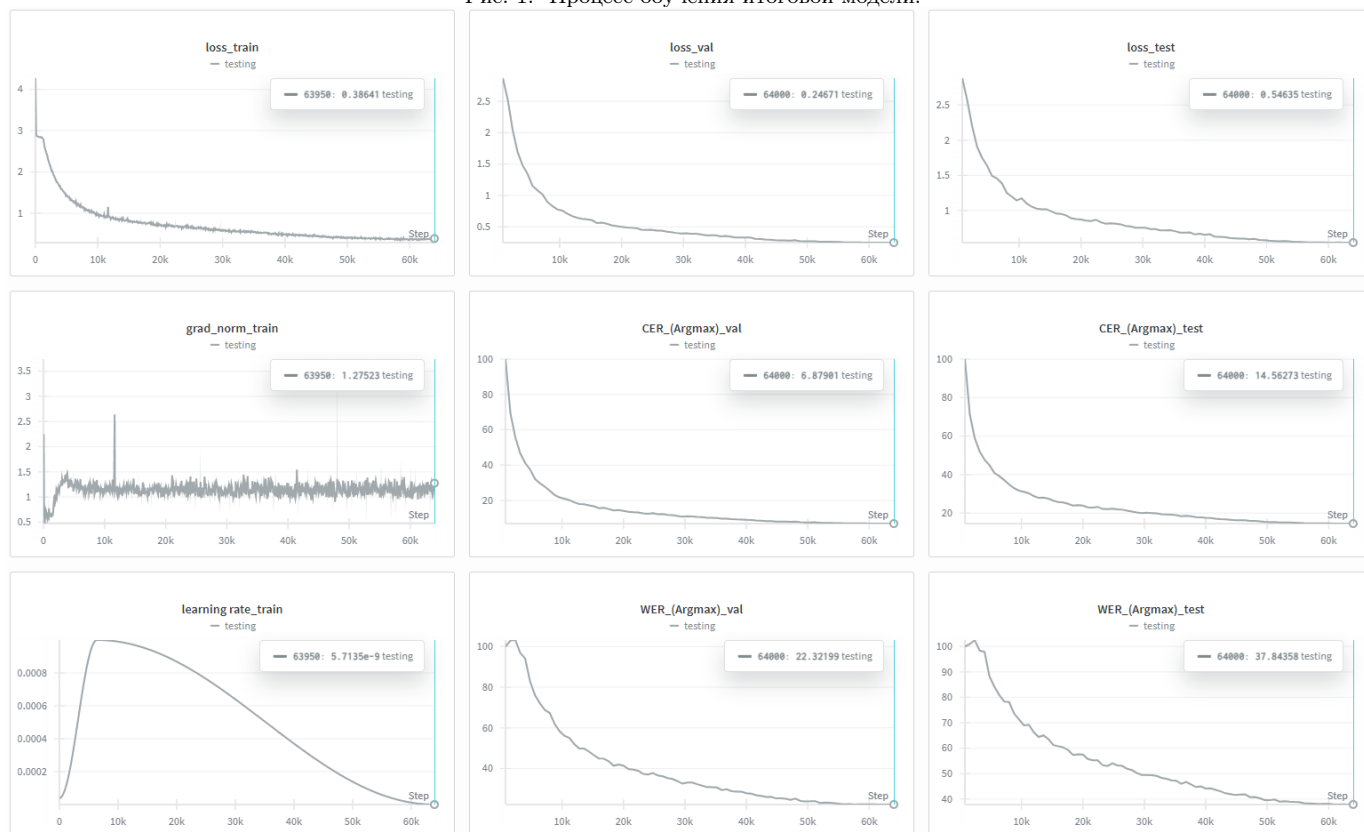


Рис. 2. Процесс обучения ВРЕ версии итоговой модели.



Рис. 3. Процесс обучения LAS модели.

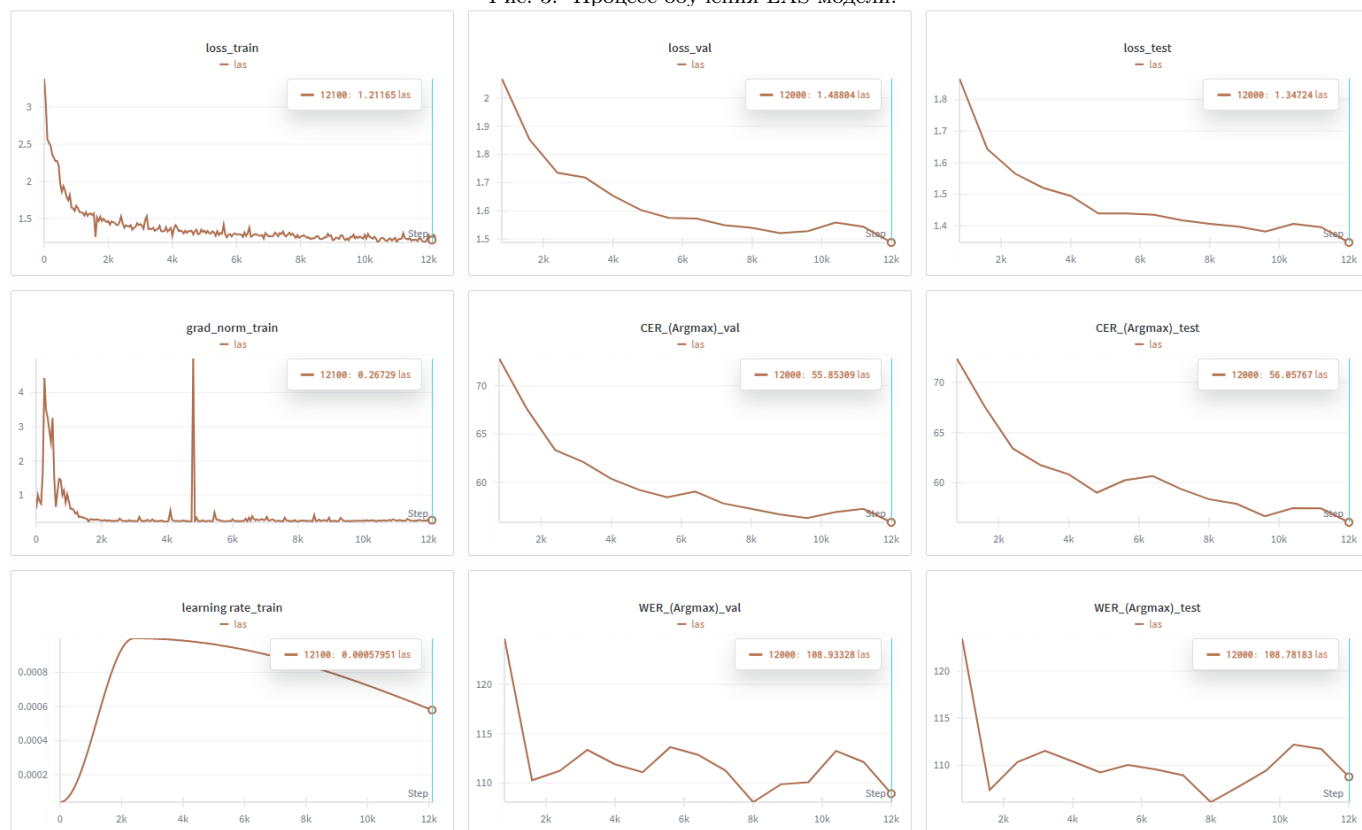


Рис. 4. Пример аугментации спектрограм.

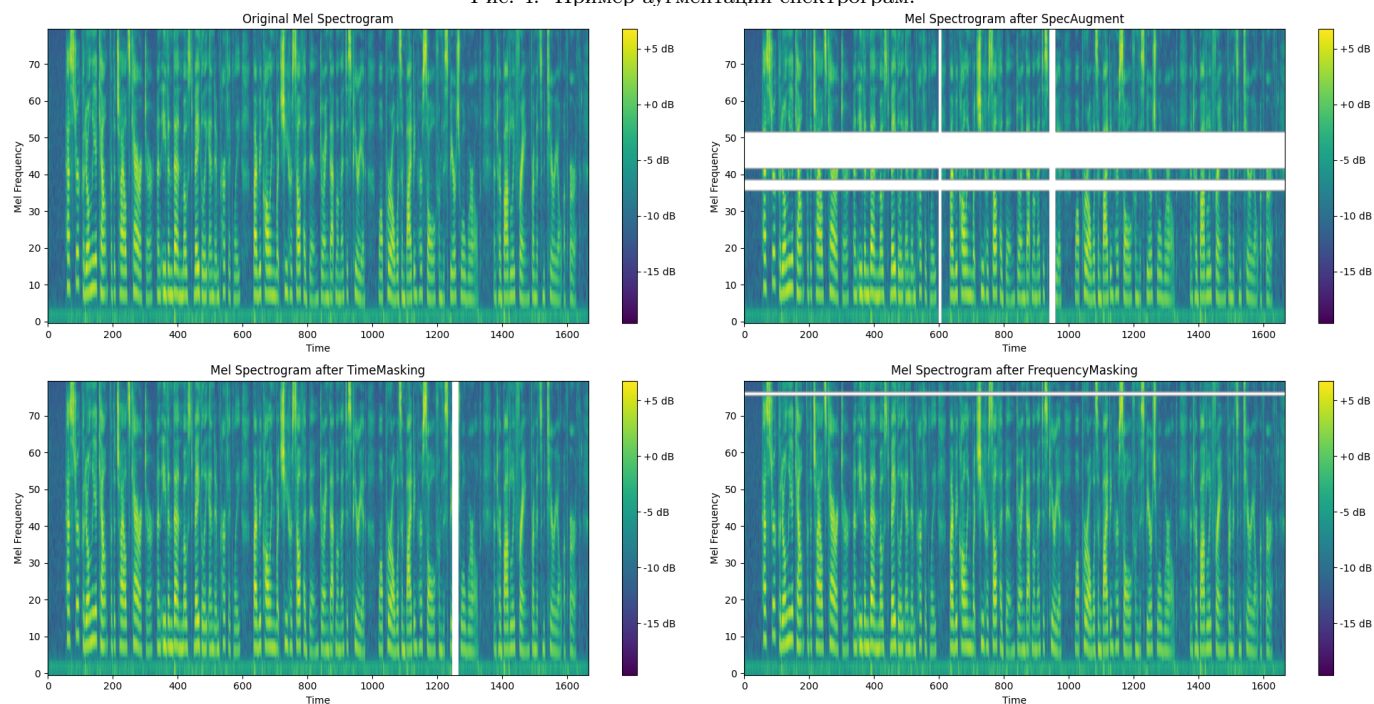


Таблица I
Итоговые значения моделей.

Model	Epochs	CER (Clean)	WER (Clean)	CER (other)	WER (other)
Conformer-CTC	80	6.87	22.32	14.56	37.84
Conformer-CTC (Beam)	—	6.80	21.95	14.32	38.02
Conformer-CTC (Beam + LM)	—	9.00	34.10	21.50	74.21
Conformer-CTC (BPE)	7	88.89	100.0	88.54	100.0
LAS	15	57.24	112.13	57.43	111.73